

Lecture 13:

Classes + Objects (Part 1)



INFO 1100: Introduction to Programming

Quiz policies

introduction to
Object-Oriented Programming

introduction to
Object-Oriented Programming

a programming paradigm that organizes
code around objects

we've learned about many data types

we've learned about many data types

float

int

bool

str

we've learned about many data types

float

int

bool

str

and many ways of organizing data

we've learned about many data types

float

int

bool

str

and many ways of organizing data

lists

dictionaries

tuples

sets

what if built-in types aren't enough?

what if built-in types aren't enough?

you guys have proposed...

what if built-in types aren't enough?

you guys have proposed...

students

recipes

courses

menu items

what if built-in types aren't enough?

you guys have proposed...

students

recipes

courses

menu items

can we make our own data types with
their own associated behavior?

Example:
points in 3D space

Example:

points in 3D space

to represent a single point: x , y , and z coordinates

Example:

points in 3D space

to represent a single point: x , y , and z coordinates



3 variables

Example:

points in 3D space

to represent a single point: x , y , and z coordinates



3 variables

what if we had a lot of points?

Example:

points in 3D space

to represent a single point: x , y , and z coordinates



3 variables

what if we had a lot of points?

x_0, y_0, z_0 and x_1, y_1, z_1 and x_2, y_2, z_2 and ...

Example:

points in 3D space

to represent a single point: x , y , and z coordinates



3 variables

what if we had a lot of points?

x_0, y_0, z_0 and x_1, y_1, z_1 and x_2, y_2, z_2 and ...

it gets very messy!

Example:

points in 3D space

to represent a single point: x , y , and z coordinates



what if we could make a single variable that
represents a point?

Example:

points in 3D space

to represent a single point: x , y , and z coordinates



what if we could make a single variable that
represents a point?

an **object**!

Warning

The code we'll learn today is correct, but not idiomatic – that is, not how most programmers would write it.

Warning

The code we'll learn today is correct, but not idiomatic – that is, not how most programmers would write it.

We'll write more idiomatic code next week!

Objects

Objects

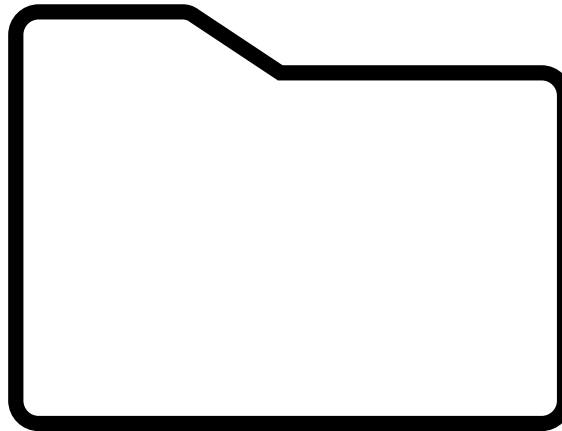
What are they?

Objects

an object is like a filing folder

Objects

an object is like a filing folder

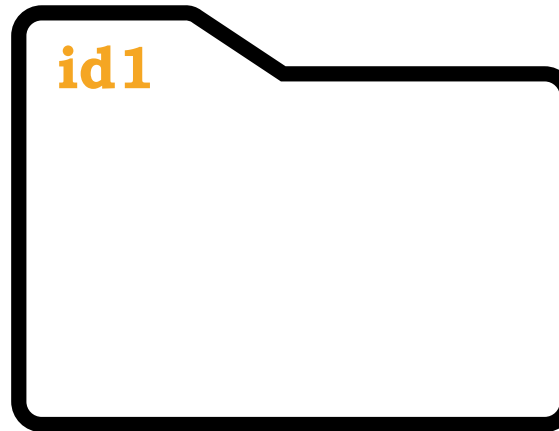


Objects

an object is like a filing folder

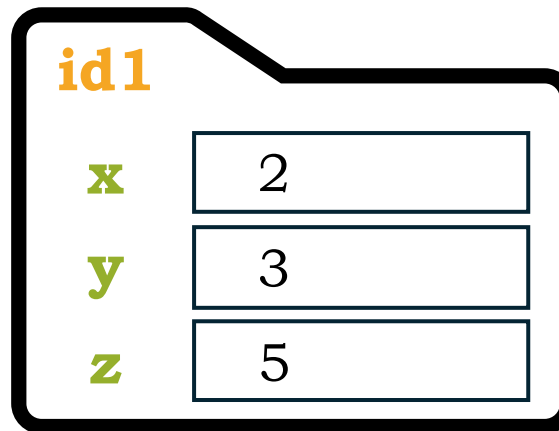
identifier

- a unique identifier assigned by Python
 - unchangeable



Objects

an object is like a filing folder



attributes

- specific variables associated with the object
- can change

but what is an object's type?

but what is an object's type?

any guesses?

but what is an object's type?

a **class**!

but what is an object's type?

a **class**!

a programmer-defined type

Classes

Classes

What are they?

Classes

let's define our own type!

Classes

```
class <class-name>:  
    """class description"""
```

Classes

```
class Point:
```

```
    """Represents a 3D point"""
```

Classes

```
class Point:
```

```
    """Represents a 3D point"""
```

how do we create an
object of this type?

Classes

```
class Point:
```

```
    """Represents a 3D point"""
```

```
p = Point()
```

Classes

```
class Point:
```

```
    """Represents a 3D point"""
```

```
p = Point()
```

use the class name like it's a function

Classes

```
class Point:
```

```
    """Represents a 3D point"""
```

```
p = Point()
```

the type of p is <class '__main__.Point'>

Classes

```
class Point:
```

```
    """Represents a 3D point"""
```

```
p = Point()
```

the type of p is <class '__main__.Point'>

__main__ is where the class is defined

Classes

```
class Point:
```

```
    """Represents a 3D point"""
```

```
p = Point()
```

creating an object is called **instantiation**

Classes

```
class Point:
```

```
    """Represents a 3D point"""
```

```
p = Point()
```

an object is an **instance** of a class

Attributes

Attributes

```
class Point:
```

```
    """Represents a 3D point"""
```

Attributes

```
class Point:
```

```
    """Represents a 3D point"""
```

```
p = Point()
```

Attributes

```
class Point:
```

```
    """Represents a 3D point"""
```

```
p = Point()
```

```
p.x = 2
```

```
p.y = 3
```

```
p.z = 5
```


Attributes

```
class Point:
```

```
    """Represents a 3D point"""
```

```
p = Point()
```

```
p.x = 2
```

```
p.y = 3
```

```
p.z = 5
```

<class-obj>.<attribute>

Attributes

```
class Point:
```

```
    """Represents a 3D point"""
```

```
p = Point()
```

```
p.x = 2
```

```
p.y = 3
```

```
p.z = 5
```

objects are mutable

Attributes

```
<class-obj>.<attribute>
```

you can also use this syntax
to access attributes

Attributes

`<class-obj>.<attribute>`

```
print (p.y)
```

Attributes

`<class-obj>.<attribute>`

```
print (p.y)
```



3

Attributes

we can use `hasattr()` to check
if an object has an attribute

Attributes

we can use `hasattr()` to check
if an object has an attribute

```
print(hasattr(p, "x"))
```

Attributes

we can use `hasattr()` to check
if an object has an attribute

```
print(hasattr(p, "x"))
```



True

Attributes

we can use `hasattr()` to check
if an object has an attribute

```
print(hasattr(p, "ab"))
```

Attributes

we can use `hasattr()` to check
if an object has an attribute

```
print(hasattr(p, "ab"))
```



False

Creating objects

Creating objects

we can write a function to create objects

Creating objects

we can write a function to create objects

```
def make_point(x, y, z):
```

Creating objects

we can write a function to create objects

```
def make_point(x, y, z):  
    p = Point()
```

Creating objects

we can write a function to create objects

```
def make_point(x, y, z):  
    p = Point()  
    p.x = x  
    p.y = y  
    p.z = z
```

Creating objects

we can write a function to create objects

```
def make_point(x, y, z):  
    p = Point()  
    p.x = x  
    p.y = y  
    p.z = z  
    return p
```


Creating objects

```
def make_point(x, y, z):  
    p = Point()  
    p.x = x  
    p.y = y  
    p.z = z  
    return p
```

Creating objects

```
def make_point(x, y, z):  
    p = Point()  
    p.x = x  
    p.y = y  
    p.z = z  
    return p
```

```
new_point = make_point(2, 3, 5)
```

Aliasing

Aliasing

like lists, objects are **aliased**

Aliasing

```
p1 = make_point(2, 3, 5)
```

Aliasing

```
p1 = make_point(2, 3, 5)
```

```
p2 = p1
```

Aliasing

```
p1 = make_point(2, 3, 5)
```

```
p2 = p1
```

```
p2.x = 99
```

Aliasing

```
p1 = make_point(2, 3, 5)
p2 = p1
p2.x = 99
```

what happens if we run:

```
print(p1.x)
```


Aliasing

```
p1 = make_point(2, 3, 5)
p2 = p1
p2.x = 99
```

what happens if we run:

```
print(p1.x)
```

Aliasing

```
p1 = make_point(2, 3, 5)
p2 = p1
p2.x = 99
```

we can return to our folder metaphor:

Aliasing

```
p1 = make_point(2, 3, 5)
p2 = p1
p2.x = 99
```

we can return to our folder metaphor:

p2 = p1 gives a different name to the same folder; it doesn't create a new one

Enter in the chat:

**What class would you like to
define?**

Object-Oriented **Vocabulary**

Object-Oriented **Vocabulary**

type: a set of possible values and associated operations

Object-Oriented **Vocabulary**

type: a set of possible values and associated operations

class: a programmer-defined type

Object-Oriented **Vocabulary**

type: a set of possible values and associated operations

class: a programmer-defined type

object: a unit that bundles data and behavior

Object-Oriented **Vocabulary**

type: a set of possible values and associated operations

class: a programmer-defined type

object: a unit that bundles data and behavior

instance: an object belonging to a particular class

Object-Oriented **Vocabulary**

type: a set of possible values and associated operations

class: a programmer-defined type

object: a unit that bundles data and behavior

instance: an object belonging to a particular class

variable: a name that refers to a value

Object-Oriented **Vocabulary**

type: a set of possible values and associated operations

class: a programmer-defined type

object: a unit that bundles data and behavior

instance: an object belonging to a particular class

variable: a name that refers to a value

attribute: a variable that belongs to an object

Example 1

```
class Account:
    """Represents a bank account"""

def make_account(owner, balance):
    a = Account()
    a.owner = owner
    a.balance = balance
    return a

acct = make_account("Sam", 100)
acct.balance += 50
acct.balance -= 30
print(acct.balance)
```

Example 2

```
class Person:
    """Represents a person"""
def make_person(name):
    p = Person()
    p.name = name
    p.langs = set()
    return p

def learn(person, lang):
    person.langs.add(lang)
```

```
ana = make_person("Ana")
learn(ana, "Python")
learn(ana, "Java")
learn(ana, "Python")
print(len(ana.langs))
print("Java" in ana.langs)
```

Example 3

```
class Contact:
    """Represents a contact"""

def make_contact(text):
    c = Contact()
    c.name = re.findall(r"Name: (.+)", text)[0]
    c.phone = re.findall(r"\d{3}-\d{4}", text)[0]
    return c

person = make_contact("Name: Bob Smith\nPhone: 555-1234")
print(person.name)
print(person.phone)
```

Example 4

```
class Point:
    """Represents a 3D point"""

def make_point(coords):
    p = Point()
    p.x, p.y, p.z = coords
    return p

pt = make_point((2, 3, 5))
print(pt.y)
print(pt.x + pt.z)
```

Questions?

(... I'll wait for 2)

For tomorrow:

(07/17 @ 11:30am)

- Lab 13
- Quiz tomorrow!

For tomorrow:
(07/17 @ 11:30am)

- Lab 13
- Quiz tomorrow!

Monday:
(07/20 @ 9:00am)

Assignment 4

Tuesday:
(07/20 @ 11:30am)

Readings + GRQs